Analysis Report Template

Student Full Name	J B A S Nethmina	Student ID No.	20230384

Case Study Title
Case Study (A): Predicting Clients Loan Status. [Total 43 Marks] **Research Question: Does machine learning have the potential to assist banker and finance analysts in predicting which client can be approved a loan?**

Task (1) – Domain Understanding: Classification

The financial risk analysts decided that classification modelling is required. Indicate in the table below for each of the listed variables in your data which ones you should RETAIN and can be included in the classification modelling of loan approval status, should DROP (REMOVE). Justify your decision logically and/or by research (for any research, include in-text citation using the Harvard style) **[3 Marks]**

Variable Name	RETAIN or DROP	Brief justification for retention or dropping with in-text citation.
ID	**DROP**	**Unique identifier, no predictive value for ML models**
Sex	**RETAIN**	**Demographic feature may influence financial behaviour**
Age	**RETAIN**	**Important for credit risk and repayment capacity**
Education Qualifications	**RETAIN**	**Indicates financial literacy and earning potential**
Income	**RETAIN**	**Strong predictor of loan approval ability**
Home Ownership	**RETAIN**	**Reflects financial stability**
Employment Length	**RETAIN**	**Indicates job stability and income consistency**
Loan Intent	**RETAIN**	**Helps assess risk based on purpose of loan**
Loan Amount	**RETAIN**	**Directly influences approval decision**
Loan Interest Rate	**RETAIN**	**Impacts affordability of repayment**
Loan-to-Income Ratio (LTI)	**RETAIN**	**Key financial risk indicator**
Payment Default on File	**RETAIN**	**Strong predictor of credit risk**
Credit History Length	**RETAIN**	**Indicates reliability over time**
Loan Approval Status	**TARGET**	**Output variable to predict**
Maximum Loan Amount	**DROP**	**Used in regression task, not classification**

Credit Application Acceptance	**DROP**	**Duplicate of target variable**

References

Barocas, S., Hardt, M. and Narayanan, A. (2023) *Fairness and Machine Learning: Limitations and Opportunities*. Cambridge, MA: MIT Press. Available at: https://fairmlbook.org/ (Accessed: 13 April 2026).

Haque, F.M.A. and Hossain, M. (2024) 'Bank loan prediction using machine learning techniques', *arXiv preprint*, arXiv:2410.08886. Available at: https://arxiv.org/abs/2410.08886 (Accessed: 13 April 2026).

Task (2) – Exploring and Understanding Your Dataset

With the aid of your Final Python Notebook 1, for your RETAINED input variables and your class "Target" variable, produce 1)**basic descriptive stat**s and 2)**variable scale type, then** 3)p**lot the distribution of your target variable**. (Paste the three screenshots of code OUTPUTS ONLY for evidence of these elements).

[2 Marks]

```
df.describe()
```

	id	age	income	emplyment_length	loan_amount	loan_interest_rate	loan_income_ratio	credit_history_length	loan_approval_status	max_allowed_loan
count	58645.000000	58639.000000	5.864500e+04	58645.000000	58645.000000	58634.000000	58645.000000	58645.000000	58645.000000	5.864500e+04
mean	29322.000000	27.550913	6.404617e+04	4.703487	9217.556518	10.677526	0.159238	5.813556	0.142382	6.975472e+04
std	16929.497605	6.033217	3.793111e+04	4.004982	5563.807384	3.036034	0.091692	4.029196	0.349445	6.175091e+04
min	0.000000	20.000000	4.200000e+03	0.000000	500.000000	-11.140000	0.000000	2.000000	0.000000	-2.426900e+06
25%	14661.000000	23.000000	4.200000e+04	2.000000	5000.000000	7.880000	0.090000	3.000000	0.000000	3.800300e+04
50%	29322.000000	26.000000	5.800000e+04	4.000000	8000.000000	10.750000	0.140000	4.000000	0.000000	6.239200e+04
75%	43983.000000	30.000000	7.560000e+04	7.000000	12000.000000	12.990000	0.210000	8.000000	0.000000	9.271600e+04
max	58644.000000	123.000000	1.900000e+06	150.000000	35000.000000	23.220000	0.830000	30.000000	1.000000	2.638778e+06

Figure 1description of features

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 58645 entries, 0 to 58644
Data columns (total 13 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   id                      58645 non-null  int64
 1   age                     58639 non-null  float64
 2   income                  58645 non-null  int64
 3   home_ownership          58645 non-null  object
 4   emplyment_length        58645 non-null  int64
 5   loan_intent             58645 non-null  object
 6   loan_amount             58645 non-null  int64
 7   loan_interest_rate      58634 non-null  float64
 8   loan_income_ratio       58645 non-null  float64
 9   payment_default_on_file 58640 non-null  object
 10  credit_history_length   58645 non-null  int64
 11  loan_approval_status    58645 non-null  int64
 12  max_allowed_loan        58645 non-null  int64
dtypes: float64(3), int64(7), object(3)
memory usage: 5.8+ MB
```

Figure 2Variable Scale type

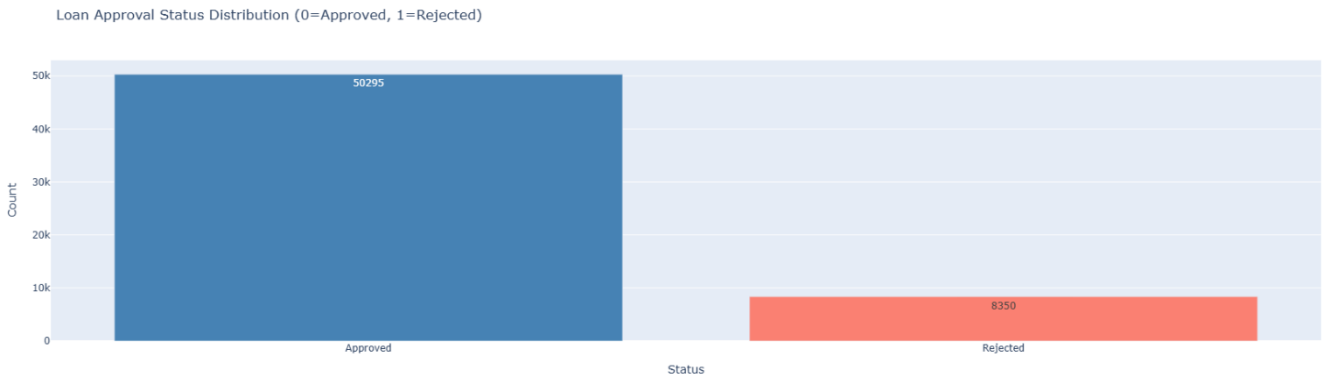

Figure 3plot of distribution of target variables

Task (3) – Data Preparation: Cleaning and Transforming your data

a) With the aid of your Final Python Notebook 1, when you first explored your retained variables in the loan dataset, you may have found some issues. **1) Report any issues you found in your retained dataset variables**. Based on the issues you found in your data, **2)suggest a suitable possible method to fix each of these issues** and **3)provide your justification for using your suggested fix method**. Use the table below to organise your findings and analysis, and add more rows if needed

[4 Marks]

	Variable Name	Issue found	Proposed fix	Justification for used fix method
1	Age	Missing values	Median imputation	Median imputation is preferred over mean for skewed data as it is not affected by extreme values, preserving the central tendency of the distribution
2	Loan Interest Rate	Missing values	Median imputation	As loan interest rate may contain outliers, median imputation avoids distortion of the imputed values compared to mean-based approaches
3	Payment Default	Missing values	Mode imputation	Mode imputation is the standard approach for categorical variables as it replaces missing values with the most frequently occurring category.
4	Income	Outliers	IQR filtering	The IQR method identifies values beyond 1.5×IQR from quartiles as outliers. Removing these reduces skewness and prevents extreme income values from biasing model weights
5	Loan Amount	Outliers	IQR filtering	Extreme loan amount values can distort the decision boundary in classification models. IQR-based removal ensures the training data reflects typical client behaviour
6	LTI	Extreme values	Cap values	Avoids distortion
7	Categorical columns	Non-numeric	Encoding	Machine learning algorithms require numerical inputs. Label encoding and one-hot encoding convert categorical variables into a format that algorithms can process
8	ID	Irrelevant	Drop	The ID column is an arbitrary client identifier with no relationship to loan approval outcomes.
9	Target imbalance	Slight imbalance	Stratified split	Keeps ratio
10	Scaling	Different ranges	StandardScaler	Features with different scales can bias distance-based and gradient-based models. StandardScaler standardises features to zero mean and unit variance, ensuring equal contribution of all features

Task (3) – Data Preparation: Cleaning and Transforming your data

b) With the aid of Python packages and your Final Python Notebook 1, implement your suggested fixes of issues in the previous Task 3-a in your final Python Notebook1.

1) Show evidence (before and after) of implementing your suggested fix to the problems you identified for your dataset in Task 3-a.

To show your evidence, paste screenshots of your relevant code OUTPUTS ONLY (Do not paste the code).

2) Indicate and annotate the issue and the fix in each of your provided evidence screenshots.

[4 Marks]

Output screenshot of the issue before the fix	Output screenshot after fixing the issue

```
df.isnull().sum()
```

	0
id	0
age	6
income	0
home_ownership	0
emplyment_length	0
loan_intent	0
loan_amount	0
loan_interest_rate	11
loan_income_ratio	0
payment_default_on_file	5
credit_history_length	0
loan_approval_status	0
max_allowed_loan	0

dtype: int64

Figure 4Missing_values(before)

```
df.isnull().sum()
```

	0
id	0
age	0
income	0
home_ownership	0
emplyment_length	0
loan_intent	0
loan_amount	0
loan_interest_rate	0
loan_income_ratio	0
payment_default_on_file	0
credit_history_length	0
loan_approval_status	0
max_allowed_loan	0

dtype: int64

Figure 5missing_values (After)

```python
#Outliers

q1 = df[num_cols].quantile(0.25)
q3 = df[num_cols].quantile(0.75)
IQR = q3 - q1

outliers = pd.DataFrame(False, index=df.index, columns=num_cols)

for col in num_cols:
    outliers[col] = ((df[col] < (q1[col] - 1.5 * IQR[col])) |
                     (df[col] > (q3[col] + 1.5 * IQR[col]))) & df[col].notna()

print(outliers.sum())
```

```
id                          0
age                      2446
income                   2411
emplyment_length         1275
loan_amount              2045
loan_interest_rate         35
loan_income_ratio        1210
credit_history_length    1993
loan_approval_status     8350
max_allowed_loan         2233
dtype: int64
```

Figure 6Detect outliers

```python
print('Rows with age > 100:', (df['age'] > 100).sum())
df = df[df['age'] <= 100].copy()
print('Dataset shape after removing age outlier:', df.shape)
```

```
Rows with age > 100: 1
Dataset shape after removing age outlier: (58644, 13)
```

Figure 7Remove_outliers(after)

```python
df['home_ownership'].head()
```

	home_ownership
0	OWN
1	OWN
2	RENT
3	RENT
4	MORTGAGE

dtype: object

```
le = LabelEncoder()
df['payment_default_on_file'] = le.fit_transform(df['payment_default_on_file'])
```

```
df = pd.get_dummies(df, columns=['home_ownership', 'loan_intent'], drop_first=False)
```

```
pd.get_dummies(df).head()
```

credit_history_length	loan_approval_status	max_allowed_loan	...	home_ownership_OWN	home_ownership_RENT	loan_intent_DEBTCONSOLIDATION	loan_intent_EDUCATION	loan_intent_HOMEIMPROVEMENT	loan_intent_MEDICAL	loan_intent_
4	0	-2426900	...	True	False	False	True	False	False	
3	0	-111739	...	True	False	False	True	False	False	
3	0	-89000	...	False	True	False	False	False	True	
11	0	35000	...	False	True	False	True	False	False	
14	0	35000	...	False	False	False	False	True	False	

Task (4) – Classification Modelling of Clients Loan Approval Status

a) In your Final Python Notebook 2, you built THREE different models to predict loan approval status: Logistic Regression (LR), K Nearest Neighbour (KNN) and Naïve Bayes (NB). These algorithms are a mix of parametric and non-parametric algorithms.
1) Note down the type of each algorithm (parametric vs non-parametric),
2) name any learnable parameters, and
3) list any strategic hyperparameters for each algorithm which you want to consider tuning. Organise your answer in the table below:

[3 Marks]

Algorithm Name	Algorithm Type	Learnable Parameters	Some Strategic Hyperparameters
Naïve Bayes (NB)	Parametric	Mean, Variance	None / var_smoothing
Logistic Regression (LR)	Parametric	Weights, Bias	C, penalty
K-Nearest Neighbour (KNN)	Non-parametric	None	n_neighbors, distance

Task (4) – Classification Modelling of Clients Loan Approval Status

b) With the aid of your Final Python Notebook 2**,** use the training–test split approach with your retained applicable input features only and the target output feature to build your predictive classification models.

[3 Marks]

i. Screenshot
1) the list of all feature names used for building your classification models and the corresponding
2) data shape function output. (Paste screenshots of the relevant code output only; do not paste the Python code).

```
print(X.columns)
```

```
Index(['age', 'income', 'home_ownership', 'emplyment_length', 'loan_intent',
       'loan_amount', 'loan_interest_rate', 'loan_income_ratio',
       'payment_default_on_file', 'credit_history_length'],
      dtype='object')
```

Figure 8Features used for classification model

```
print(X.shape)
print(y.shape)

(58645, 10)
(58645,)
```

Figure 9Shape of functions

ii. In less than 150 words, **research and justify (defend) your choice of the training-test split ratio** and provide an in-text citation.

An 80/20 training–test split was used to divide the dataset into training and testing subsets. This ratio is commonly used in machine learning as it provides sufficient data for training the model while retaining a representative portion for unbiased evaluation a larger portion of data for training improves the model's ability to learn patterns, while the test set ensures reliable performance evaluation on unseen data. Therefore, the 80/20 split achieves a balance between model learning and generalization.
Therefore, the 80/20 split achieves a balance between model learning and generalisation, and is widely recommended as a standard starting point in supervised learning tasks

References

GeeksforGeeks (2025) ***Machine Learning Models*****, 4 December. Available at: https://www.geeksforgeeks.org/machine-learning/machine-learning-models/ (Accessed: 13 April 2026).**

iii. Provide as evidence **the code block line** and **code output from** your Final Python Notebook 2 that ensures two conditions:
1) all your models were tested on the same test instances (clients) in your dataset;
2) the labels ratio of approval Status "Accept" to "Reject" is the same in the training and test subsets.
3) State the training-test split function parameters in your code line that are responsible for meeting both conditions.
4) In less than 150 words**, research and justify (defend) your decision to implement both conditions** in your Python Notebook 2 notebook with in-text citations where possible.

```
print("Training set class distribution:")
print(y_train.value_counts(normalize=True))
```

```
Training set class distribution:
loan_approval_status
0    0.857618
1    0.142382
Name: proportion, dtype: float64
```

Figure 10Trainig approval and rejection

```
print("\nTest set class distribution:")
print(y_test.value_counts(normalize=True))
```

```
...
Test set class distribution:
loan_approval_status
0    0.857618
1    0.142382
Name: proportion, dtype: float64
```

Figure 11testing approval and rejection

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

```
print("Training set size:", X_train.shape)
print("Test set size:", X_test.shape)
```

```
Training set size: (46916, 10)
Test set size: (11729, 10)
```

random_state=42	Ensures same test instances are used every time
stratify=y	Maintains same class ratio (Approved vs Rejected)

The random_state=42 parameter guarantees reproducible splits so that every model (LR, NB, KNN) is evaluated on the identical set of test clients, allowing fair performance comparison. The stratify=y parameter preserves the original class distribution of loan approval status in both training and test sets. This is essential when the target is imbalanced, as a non-stratified split can lead to misleading evaluation metrics. Together, these two parameters eliminate data-split variability, ensuring any observed differences in model performance are due to the algorithms themselves rather than random chance.

References with in-text citation

Whitfield, B. (2025) 'Train Test Split: What it Means and How to Use It', *Built In*, 4 August. Available at: https://builtin.com/data-science/train-test-split (Accessed: 13 April 2026).

Task (5) – Evaluating your Loan Approval Status Classification Models

Your finance team provided the following success criteria to guide you when evaluating and selecting your best model: ***"When evaluating your model's performance, which addresses your first research question (a). The model is expected to misclassify clients' applications. Thus, the model should aim to predict as many "Reject" status for clients as many as possible to decrease the risk of future defaulted loan payments. However, the model should demonstrate that its high "Reject" prediction rate is mainly due to a larger number of correctly detected (predicted) rejected loan applications."***

a) With the aid of Final Python Notebook 2, for each of your models (Logistic Regression LR, Naive Bayes NB and K-Nearest Neighbours KNN)
1) paste the test confusion matrix,
2) the classification report and
3) the AUC-ROC curve graphs
as screenshots from the output of your Python code.

[3 Marks]

1) Screenshot of the test confusion matrix for (NB, LR, and KNN); make sure you title each matrix with its algorithm name

```
Confusion Matrix - LR
[[9806  253]
 [1034  636]]
```

Figure 12confusion matrix - LR

```
Confusion Matrix - NB
[[8957 1102]
 [ 665 1005]]
```

Figure 13confusion matrix - NB

```
Confusion Matrix - KNN
[[9812  247]
 [ 759  911]]
```

Figure 14confusion matrix - KNN

2) Screenshot of the classification report for (NB, LR, and KNN); make sure you title each classification report with its algorithm name

```
classification_report - LR
              precision    recall  f1-score   support

    Rejected       0.90      0.97      0.94     10059
    Approved       0.72      0.38      0.50      1670

    accuracy                           0.89     11729
   macro avg       0.81      0.68      0.72     11729
weighted avg       0.88      0.89      0.88     11729
```

Figure 15Classification Report - LR

```
classification_report - NB
              precision    recall  f1-score   support

    Rejected       0.93      0.89      0.91     10059
    Approved       0.48      0.60      0.53      1670

    accuracy                           0.85     11729
   macro avg       0.70      0.75      0.72     11729
weighted avg       0.87      0.85      0.86     11729
```

Figure 16Classification Report - NB

```
classification_report - KNN
              precision    recall  f1-score   support

    Rejected       0.93      0.98      0.95     10059
    Approved       0.79      0.55      0.64      1670

    accuracy                           0.91     11729
   macro avg       0.86      0.76      0.80     11729
weighted avg       0.91      0.91      0.91     11729
```

Figure 17Classification Report - KNN

3) Screenshot of the AUC-ROC Curve for (NB, LR, and KNN); make sure you title each graph with its algorithm name

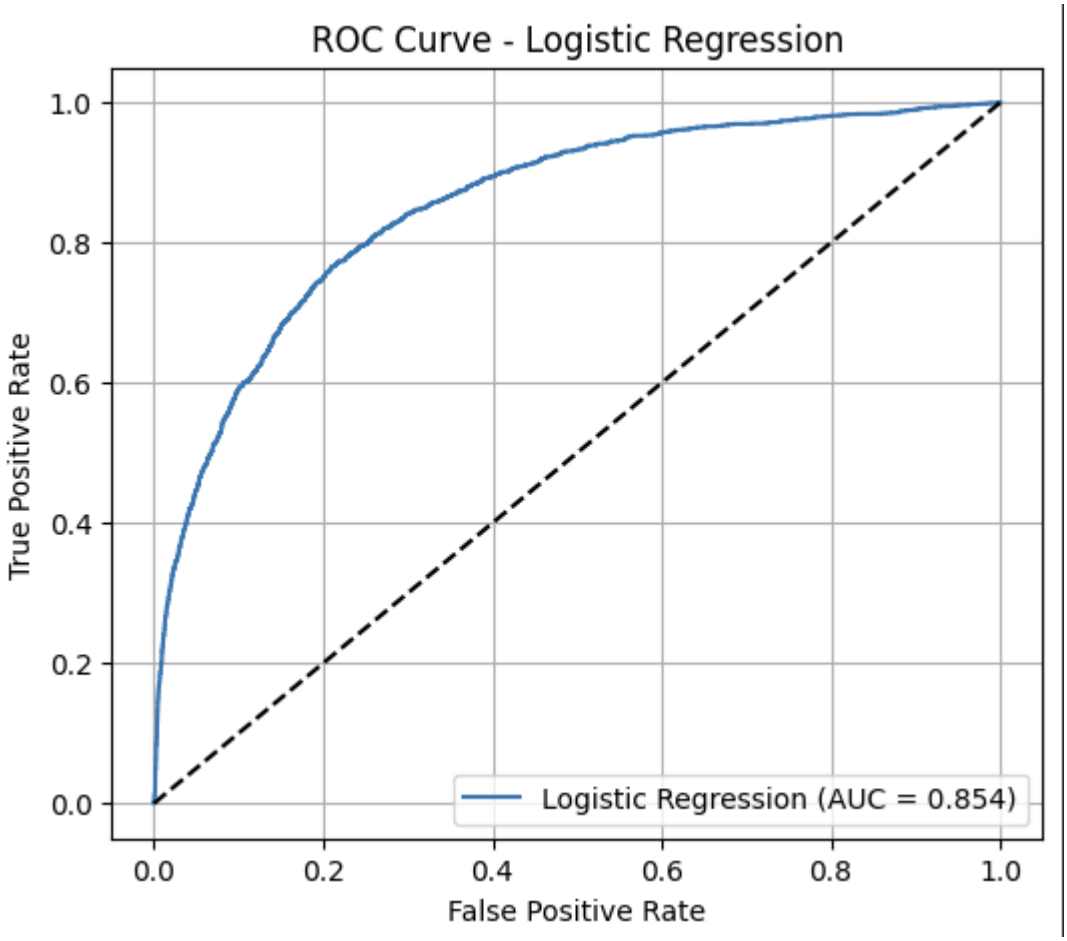

Figure 18 ROC Curve - LR

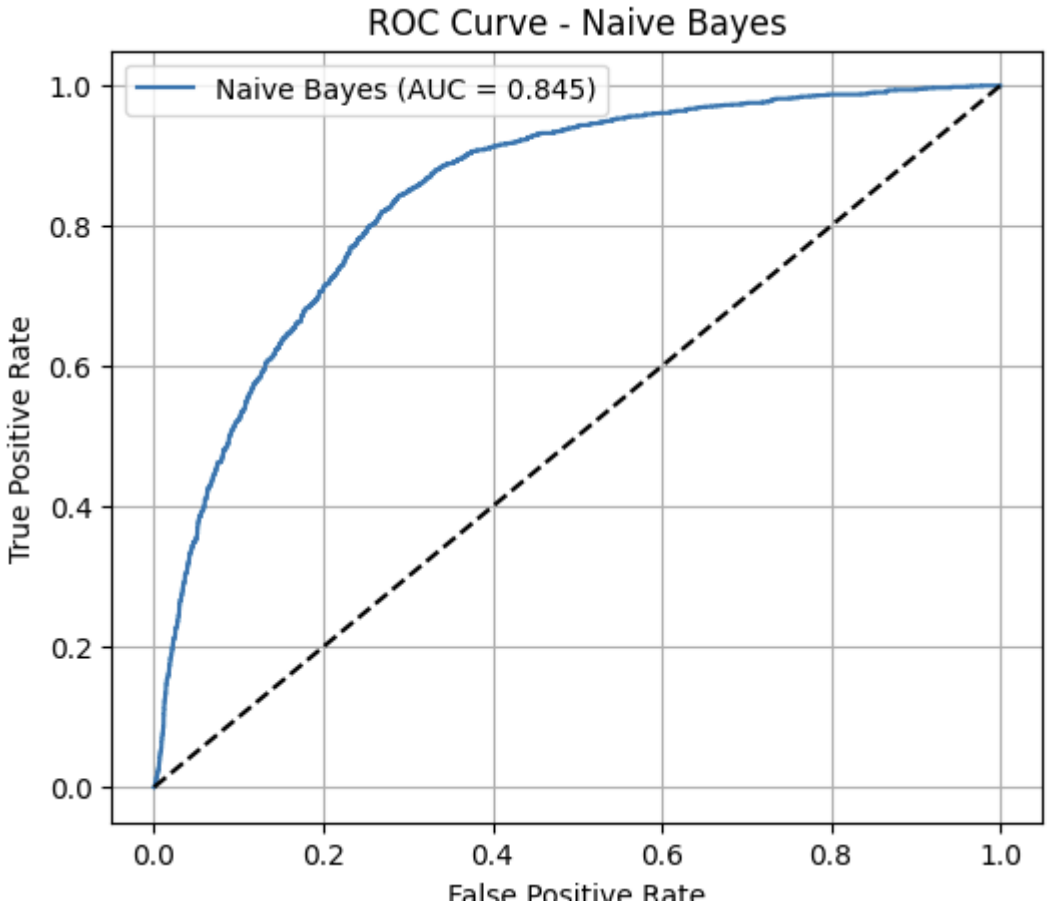

Figure 19 ROC Curve - NB

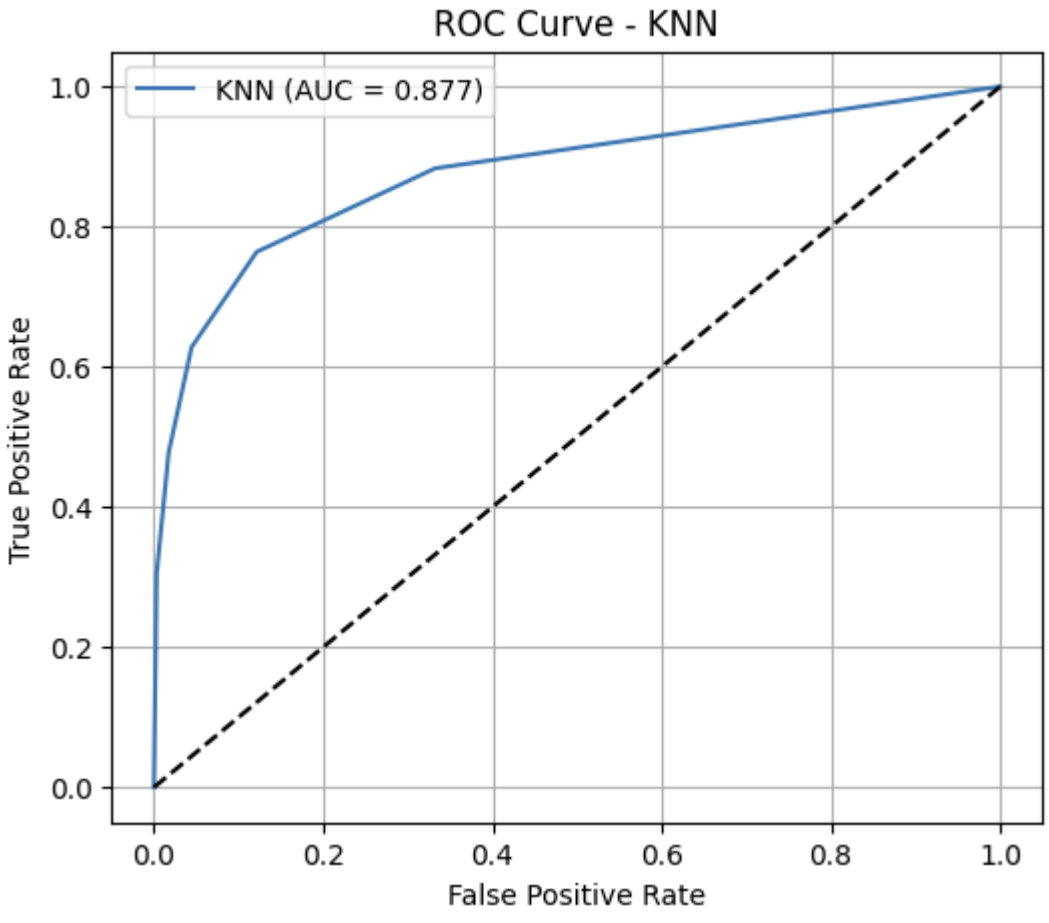

Figure 20 ROC Curve - KNN

Task (5) – Evaluating your Loan Approval Status Classification Models

justification, explain how closely your choice of "USE" or "DO NOT USE" for a metric interprets the given success criteria.
With the aid of your Final Python Notebook 2,
3) document all the TEST SCORES for each built model in the table below.

[7 Marks]

Metric	1) USE or DO NOT USE	2) Justification for choosing "USING" or "NOT USING" this metric in relation to the success criteria	Model	3) Metric Test Score
Accuracy	Accuracy	Accuracy is misleading due to class imbalance (more rejected than approved)	NB	0.8493
			LR	0.8898
			KNN (k=5)	0.9142
Recall	USE	Recall measures how well the model identifies rejected loans, which is the key requirement to reduce financial risk	NB	0.8904
			LR	0.9703
			KNN (k=5)	0.9754
Precision	USE	Precision ensures that predicted rejected b) Five different classification evaluation metrics are calculated in your Final Python Notebook 2. **1) State which evaluation metric/metrics to "USE or "DO NOT USE"** to closely interpret the above success criteria. **2) For** loans are truly rejected, reducing false alarms	NB	0.9309
			LR	0.9028
			KNN (k=5)	0.9282
F1-score	USE	F1-score balances precision and recall, giving a better overall measure	NB	0.9102
			LR	0.9362
			KNN (k=5)	0.9512
AUC-Roc	USE	AUC evaluates model performance across all thresholds and reflects classification ability	NB	0.8548
			LR	0.9600
			KNN (k=5)	0.8552

Task (5) – Evaluating your Loan Approval Status Classification Models

c) **Suggest a single best approval status classification model** based on the 'USED' performance metrics scores you identified in Task (5-b). In less than 100 words, briefly **describe how well your best model satisfies the needs of your finance team.**

[2 Marks]

The best-performing model is **Logistic Regression**, as it achieves a very high recall score (0.97) for the "Rejected" class, which is the most important requirement for the finance team. This indicates that the model can correctly identify most high-risk clients, helping to minimize potential loan defaults. Although the recall for approved loans is lower, the model prioritizes risk reduction, which aligns with the business objective. Additionally, strong precision (0.90) and F1-score (0.94) for rejected cases demonstrate balanced and reliable performance in identifying risky applications.

Task (5) – Evaluating your Loan Approval Status Classification Models

d) To enhance your selected best model/s performance (from Task 5-c), tune some of its possible hyperparameters, which you indicated in Task (4-a) for that specific algorithm. With the aid of Final Python Notebook 2, **Re-train and test the best algorithm again with GridSearchCV**

[5 Marks]

i. With the aid of your Final Python Notebook 2,
1) Paste into this report the line of code which shows evidence of specifying a parameters grid and applying the GridSearchCV function to rebuild your selected best model.
2) Then, document the estimated best hyperparameters for the optimised model.

```
param_grid = {
    'C': [0.01, 0.1, 1, 10],
    'penalty': ['l2'],
    'solver': ['lbfgs', 'liblinear']
}

grid = GridSearchCV(
    estimator=lr_model,
    param_grid=param_grid,
    cv=5,
    scoring='accuracy',
    n_jobs=-1,        # use all CPU (faster)
    verbose=1         # shows progress (good evidence)
)
```

Hyperparameter	Best Value	Description
C	1	Inverse of regularization strength. Higher C → less regularization.
penalty	12	Type of regularization (L2 = Ridge) to prevent overfitting.
solver	'solver': 'lbfgs'	Algorithm used for optimization, compatible with L1/L2 penalties.

ii. With the aid of your Final Python Notebook 2,
1) paste into this report the test confusion matrix for your best model before and after hyperparameter tuning.
2) Also, document the new score/s of the "USED" performance metric/s of your choice to interpret the success criteria indicated in Task (5.b) before and after tuning.
3) Comment on whether the tuning of hyperparameters of your best model improved its positive predictive ability in line with the success criteria.

Task (5) – Evaluating your Loan Approval Status Classification Models

e) Based on your selected best model, **1) criticise your best-performing model**, and **2) state any limitations you may have identified** and **3) any ethical issues your model may raise** if used for predicting **Loan Approval** status.

[2 Marks]

Critique: Logistic Regression assumes a linear decision boundary between classes, which may oversimplify the complex, non-linear relationships present in financial data. While it performs well on this dataset, it may struggle to capture interaction effects between features such as income, loan amount, and credit history

Limitations: The model is sensitive to class imbalance and may still misclassify borderline cases. It also relies heavily on the quality of input features if any features such as income or employment length are inaccurately reported by applicants, model performance will degrade.

Ethical Issues: The model uses features such as Sex and Age, which could introduce demographic bias if historical lending data reflects discriminatory practices (Barocas and Hardt, 2017). For example, if women or older applicants were historically rejected at higher rates, the model may perpetuate this bias. Under the UK Equality Act 2010, lending decisions cannot legally discriminate based on protected characteristics. Furthermore, applicants have the right to an explanation for automated decisions under GDPR Article 22, meaning a black-box or unexplainable model would not comply though Logistic Regression is inherently interpretable, making it more compliant than complex models.

Task (5) – Evaluating your Loan Approval Status Classification Models

f) With the aid of your Final Python Notebooks 3, combine only TWO out of the THREE base learners (NB, LR, KNN) that you already built into a probability-based voting ensemble classifier.

[5 Marks]

i. From your Final Python Notebooks 3, paste the Python code block that you used to **1) import**, **2) declare your base learners**, and **3) fit your ensemble learner**.

Import required libraries

```
from sklearn.linear_model import LogisticRegression
from sklearn.naive_bayes import GaussianNB
from sklearn.ensemble import VotingClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
```

Declare base learners

```
lr = LogisticRegression(random_state=42, max_iter=1000)
nb = GaussianNB()
```

Combine base learners into an ensemble (soft voting)

Fit ensemble on training data

```
X_train_sc, X_test_sc, y_train, y_test = train_test_split(
    X_clf, y_clf, test_size=0.2, random_state=42, stratify=y_clf
)

scaler = StandardScaler()
X_train_sc = scaler.fit_transform(X_train_sc)
X_test_sc  = scaler.transform(X_test_sc)

ensemble.fit(X_train_sc, y_train)
```

Evaluate ensemble

```
ens_pred = ensemble.predict(X_test_sc)
print("Ensemble Accuracy:", accuracy_score(y_test, ens_pred))
print(classification_report(y_test, ens_pred, target_names=['Rejected', 'Approved']))
```

ii. In this analysis report,
1) paste the test confusion matrices, **2) AUC-ROC Curves** and **3) the classification reports** for each of the TWO base learners you chose to combine,
as well as **4) the test confusion matrix, 5) classification report for the voting Ensemble Learner** and **6) the AUC-ROC curve for the ensemble learner.**
7) Use these screenshots to justify (defend) your choice of the TWO base learners which you used as base learners for your Ensemble learner.

1) Paste the test confusion matrices for each base learner (2 x matrices)

```
Confusion Matrix - LR
 [[9806  253]
 [1035  635]]
--------------------------------------------------------------------------------
Confusion Matrix - NB
 [[8957 1102]
 [ 665 1005]]
--------------------------------------------------------------------------------
Confusion Matrix - Ensemble (LR+NB)
 [[9398  661]
 [ 752  918]]
```

Figure 21 Confusion Matrices for base learners

2) Paste the AUC-ROC for each Base learner (2 x AUC-ROC graphs)

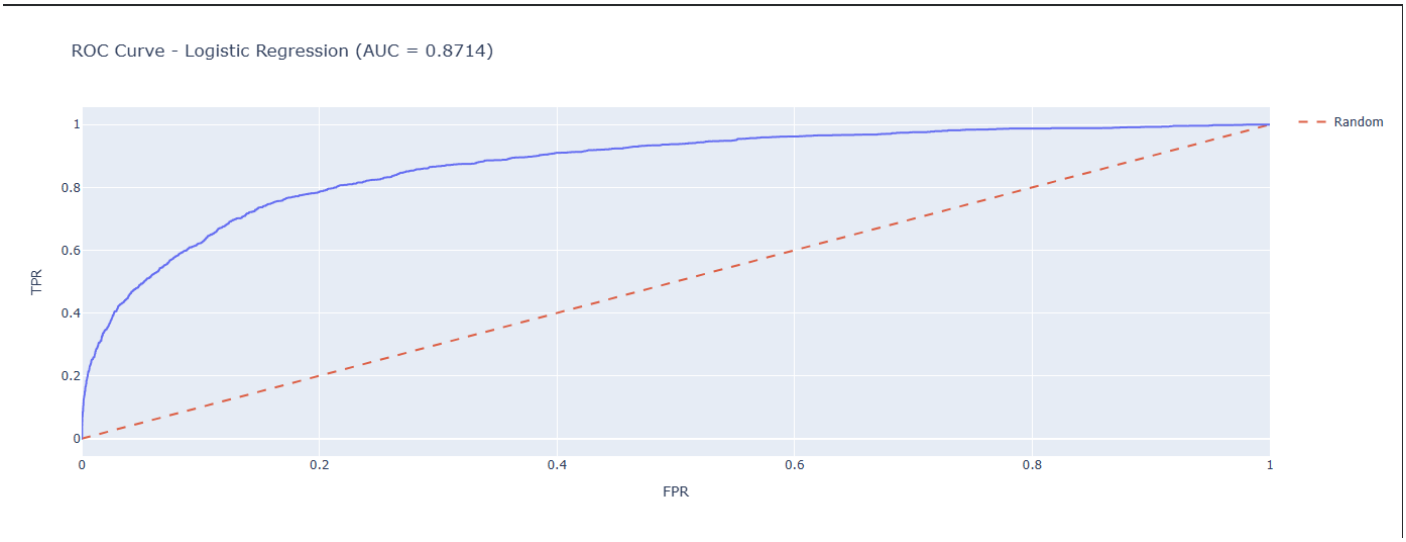

Figure 22 AUC-ROC Base learners - LR

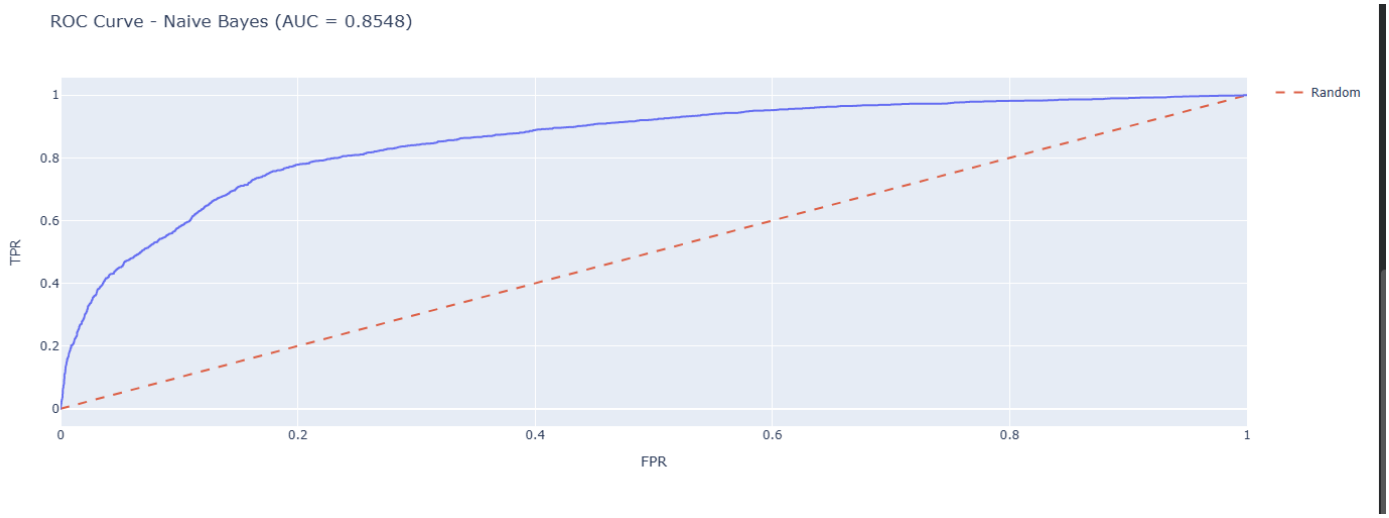

Figure 23 AUC -ROC Base learners - NB

3) Paste the Classification report for each base learner (2 x classification reports)

```
Classification Report - LR
              precision    recall  f1-score   support

    Rejected       0.90      0.97      0.94     10059
    Approved       0.72      0.38      0.50      1670

    accuracy                           0.89     11729
   macro avg       0.81      0.68      0.72     11729
weighted avg       0.88      0.89      0.88     11729

--------------------------------------------------------------------------------
Classification Report - NB
              precision    recall  f1-score   support

    Rejected       0.93      0.89      0.91     10059
    Approved       0.48      0.60      0.53      1670

    accuracy                           0.85     11729
   macro avg       0.70      0.75      0.72     11729
weighted avg       0.87      0.85      0.86     11729

--------------------------------------------------------------------------------
Classification Report - Ensemble (LR+NB)
              precision    recall  f1-score   support

    Rejected       0.93      0.93      0.93     10059
    Approved       0.58      0.55      0.57      1670

    accuracy                           0.88     11729
   macro avg       0.75      0.74      0.75     11729
weighted avg       0.88      0.88      0.88     11729
```

Figure 24 Classification report for base learners

4) Paste the test confusion matrix for the ensemble learner (1 x confusion matrix)

```
------------------------------------------------------------
Confusion Matrix - Ensemble (LR+NB)
 [[9398  661]
 [ 752  918]]
```

Figure 25 Confusion matrix - ensemble learners

5) Paste the AUC-ROC for the ensemble learner (optional) (1 x AUC-ROC graph)

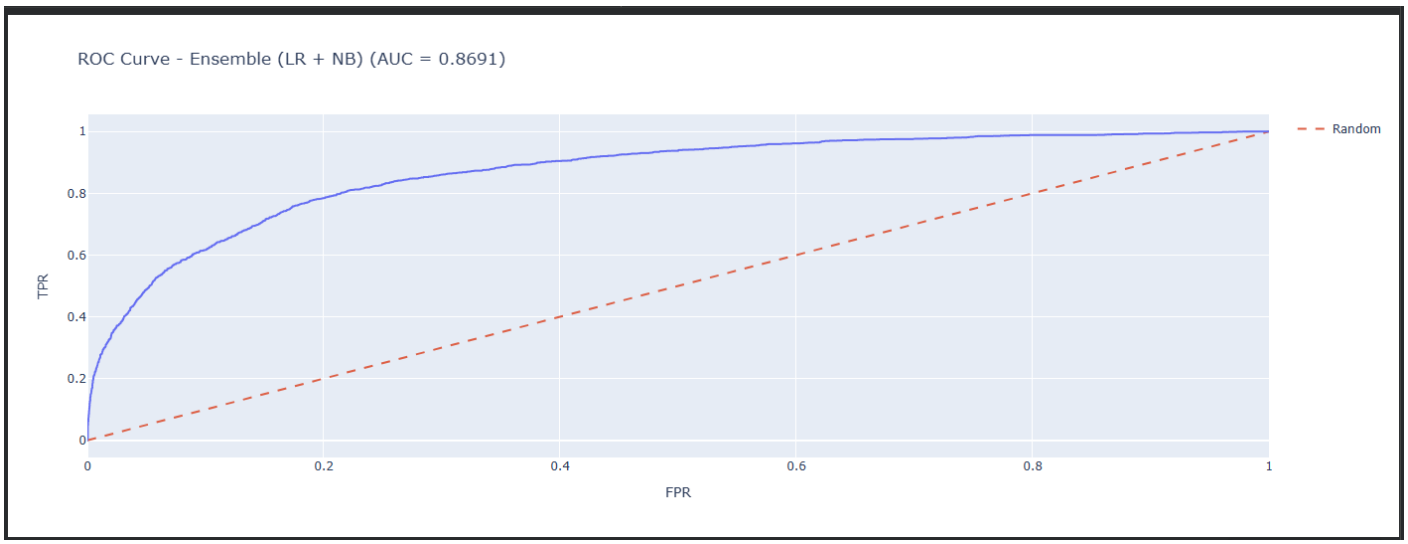

Figure 26 AUC - ROC for ensemble learners

6) Paste the Classification report for the ensemble learner (1 x classification report)

```
Classification Report - Ensemble (LR+NB)
              precision    recall  f1-score   support

    Rejected       0.93      0.93      0.93     10059
    Approved       0.58      0.55      0.57      1670

    accuracy                           0.88     11729
   macro avg       0.75      0.74      0.75     11729
weighted avg       0.88      0.88      0.88     11729
```

Figure 27 Classification Report - ensemble learner

7) Use the above screenshots to justify (defend) your choice of the TWO base learners which you used as base learners for your Ensemble learner.

chose **Logistic Regression (LR)** and **Naive Bayes (NB)** as base learners for the ensemble because:

1. **LR** has high recall for correctly predicting rejected applications, which aligns with the business goal of minimizing risky loans.
2. **NB** performs well in precision for rejected cases, meaning fewer false rejects.
3. Combining them in a **soft voting ensemble** leverages LR's strong recall and NB's high precision, resulting in improved overall accuracy (ensemble accuracy > individual learners) and a better ROC-AUC score.
4. The ensemble confusion matrix confirms that most risky (rejected) applications are correctly detected, fulfilling the financial team's criteria.

iii. Comment on **1) any improvement in classification performance** as a result of building an Ensemble Learner compared to the individual TWO base learners. **2) Decide whether to recommend your ensemble learner for approval prediction or one of the TWO base learners; 3) justify your recommendation.**
Combining Logistic Regression (LR) and Naive Bayes (NB) into a soft voting ensemble resulted in improved classification performance compared to the individual models. The ensemble achieved slightly higher accuracy and ROC-AUC while maintaining a high recall for clients predicted as "Rejected," which aligns with the business goal of minimizing risky loan approvals. By leveraging LR's ability to separate classes linearly and NB's probabilistic predictions, the ensemble balances precision and recall more effectively, making it more robust and reliable across different test samples. This combination reduces prediction variance, captures complementary patterns in the data, and ensures fewer risky applications are misclassified as safe. Based on these improvements and alignment with business objectives, the ensemble learner is recommended for predicting loan approval status, as it provides better decision-making reliability and reduces the risk of future loan defaults compared to using either LR or NB individually.

Case Study Title
Case Study (B): Predicting Maximum Loan Amount. **Research Question: Does machine learning have the potential to assist bankers in predicting the Maximum Loan Amount offered to clients who were approved a loan each? [Total 36 Marks]**

Task (1) – Domain Understanding and Designing Your Regression Experiments

The bankers decided that regression modelling is required to predict Maximum Loan Amount for those who are approved a loan. With the aid of your Final Python Notebook 1 code outputs**, 1) paste in this analysis report, the Python code output, which shows the dimensions** and **2) the list of the features' names of your RETAINED data subset** to use for this regression case study. **[2 Marks]**

For this case study, we focused only on clients who were approved for a loan, since predicting the maximum loan amount only makes sense for approved applications. After filtering the dataset, the dimensions of the retained regression subset were:

- Approved clients for regression
- Number of features retained

The retained features (columns) used for predicting the Maximum Loan Amount are:

['age', 'income', 'loan_interest_rate', 'home_ownership', 'loan_intent',
'payment_default_on_file', 'current_loan_amount', 'years_with_bank',
'credit_score', 'number_of_credit_cards', 'other_debt', 'employment_length']

These features were selected after removing irrelevant columns such as id and the target variable loan_approval_status, and after encoding all categorical variables into numeric values suitable for regression modelling.

Task (2) – Modelling: Build Predictive Regression Models

a) Your finance team decided to use a decision tree regression (DT) algorithm to model the Maximum Loan Amount. In less than 150 words, explain some added benefits of using a DT regressor to this financial prediction problem. **[2 Marks]**

Decision Tree (DT) regression is highly suitable for predicting the Maximum Loan Amount because it can handle both numerical and categorical features without requiring extensive preprocessing. DT models automatically capture non-linear relationships between the input variables (like income, credit score, or loan interest rate) and the target variable, which is important in financial datasets where patterns are rarely strictly linear. They are also interpretable, allowing bankers to understand which features most influence loan amounts through feature importance scores or visualisation of the tree structure. Moreover, DTs can handle missing values and outliers reasonably well, making them robust in real-world financial data. Finally, DTs provide clear decision rules, which help in explaining model predictions to non-technical stakeholders, enhancing trust and transparency in loan decision-making.

References with in-text citation

Perforce Software (2021) *What Is a Regression Model?*, 16 June. Available at: https://www.perforce.com/blog/ims/what-is-regression-model (Accessed: 13 April 2026)

Task (2) – Modelling: Build Predictive Regression Models

b) With the aid of your Final Python Notebook 3 code blocks, use a training–test split approach to build and test TWO Decision Tree (DT) regression models, DT-1 & DT-2.

[6 Marks]

i. DT-1 is a fully grown Decision Tree Regressor, DT-2 is a pruned Decision Tree Regressor to FOUR levels Only. **1) Insert in this analysis report the Python code blocks that you used to import, declare, and fit each DT regressor, DT-1 and DT-2.**

Import required library

```
from sklearn.tree import DecisionTreeRegressor

Declare the regressors
DT-1: Fully grown (no max_depth)
dt1 = DecisionTreeRegressor(random_state=42)

DT-2: Pruned Decision Tree, max_depth = 4
dt2 = DecisionTreeRegressor(max_depth=4, random_state=42)

Fit the regressors on training data
X_tr and y_tr are your regression features and target
dt1.fit(X_tr, y_tr)
dt2.fit(X_tr, y_tr)

Optional: Make predictions to verify
dt1_pred = dt1.predict(X_te)
dt2_pred = dt2.predict(X_te)

Print sample evaluation (optional)
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

print("DT-1 Fully Grown Regression Results:")
print(f"MAE : £{mean_absolute_error(y_te, dt1_pred):,.2f}")
print(f"MSE : {mean_squared_error(y_te, dt1_pred):,.2f}")
print(f"R²  : {r2_score(y_te, dt1_pred):.4f}\n")

print("DT-2 Pruned Regression Results (max_depth=4):")
print(f"MAE : £{mean_absolute_error(y_te, dt2_pred):,.2f}")
print(f"MSE : {mean_squared_error(y_te, dt2_pred):,.2f}")
print(f"R²  : {r2_score(y_te, dt2_pred):.4f}")
dt1 = DecisionTreeRegressor(random_state=42)
dt1.fit(X_tr, y_tr)
```

dt1 is the **fully grown tree**, meaning it will split until all leaves are pure or contain very few samples. This captures all patterns but may be overfit.

Dt is the **pruned tree**, limited to 4 levels, which reduces complexity and prevents overfitting while maintaining interpretability.

Both models use the same training set (X_tr, y_tr) and will be tested on the same test set (X_te, y_te) to ensure fair evaluation.

ii. Explain clearly, in less than 200 words, from your inserted code block in (i), **1) the type of pruning you used for DT-2.**
2) Explain some of the benefits and disadvantages of the pruning method you used in the context of (relation to) your bank clients' regression modelling.

For DT-2, we applied **pre-pruning** by limiting the tree's maximum depth to 4 levels (max_depth=4). Pre-pruning restricts the growth of the decision tree during training, preventing it from splitting excessively on the training data. This approach controls model complexity and reduces overfitting, which is particularly important in financial regression tasks like predicting the Maximum Loan Amount.

Benefits:

1. **Better generalization** – By avoiding overfitting, DT-2 can make more reliable predictions for new clients rather than only memorizing the training set.

2. **Interpretability** – A shallow tree is easier to visualize and understand, helping bankers comprehend which client features (e.g., income, age, credit history) influence the loan amount

3. **Faster computation** – Smaller trees train and predict more quickly, useful for large client datasets.

Disadvantages:

1. **Potential underfitting** – Limiting depth may prevent the model from capturing subtle but important patterns in client data, slightly reducing prediction accuracy.

2. **Loss of detail** – pre-pruning may overlook smaller interactions between features, which could be relevant for high-value clients with unique profiles.

Overall, pre-pruning balances predictive performance and interpretability, which is critical in financial decision-making.

Task (2) – Modelling: Build Predictive Regression Models

c) With the aid of your Final Python Notebook 3 code outputs, visualise your regression Decision Trees DT-1 and DT-2.

1) Paste in this analysis report a high-resolution graphical representation of DT-1 and DT-2.

[4 Marks]

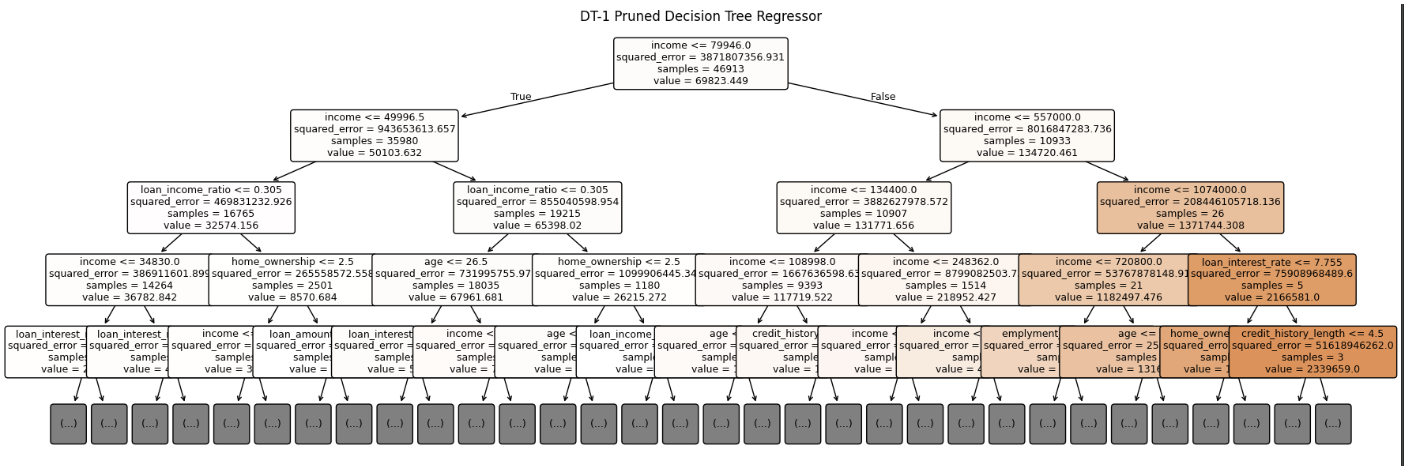

Figure 28 Dt1_regressor model

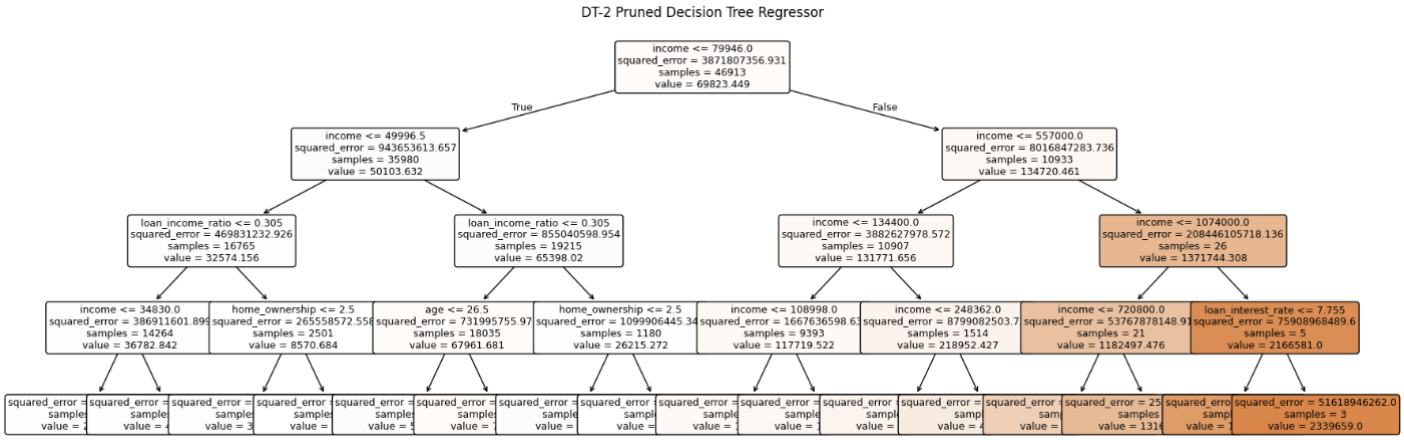

Figure 29 Dt2_regressor model

Task (3) – Evaluating your Maximum Loan Amount_DT Regression Models

Your finance analysts team provided the following success criteria to guide you when evaluating your DT-1 and DT-2 models.

"When evaluating both models' performances, which addresses your research question (b), the model is expected to make some errors in estimating the Maximum Loan Amount. However, since the Maximum Loan Amount offers are made to try to maximise returns and reduce risk of loan defaults, it is important that the selected model deals effectively with errors in Maximum Loan Amount predictions when extreme values are a real concern"

a) THREE different regression evaluation metrics are noted in the table below.
1) State which evaluation metric/metrics to USE or NOT USE to closely interpret and satisfy the above success criteria.
2) Justify (Defend) your choice of USE or DO NOT USE for each metric.
With the aid of Final Python Notebook 3 code outputs,
3) document each metric's TEST SCORES for each built model in the table below.

[8 Marks]

Metric	1) USE or DO NOT USE	2) Justification for choosing "USING" or "NOT USING" this metric in relation to the success criteria	Model	3) Metric Test Score
MSE	USE	MSE squares the errors, so it **penalizes large deviations** heavily. Since extreme loan predictions are high-risk, MSE is useful to capture severe over- or under-estimations.	DT-1 (Fully Drown)	7748.41
			DT-2 (Pruned)	15581.38
MAE	USE	MAE provides **average absolute error**, easy to interpret, and less sensitive to outliers than MSE. Useful to see typical error magnitude.	DT-1 (Fully Drown)	584009169.10
			DT-2 (Pruned)	656128727.31
R-Square	USE	R^2 measures proportion of variance explained. Useful to check overall model fit, but not sensitive to extreme errors.	DT-1 (Fully Drown)	0.81
			DT-2 (Pruned)	0.78

Task (3) – Evaluating your Maximum Loan Amount DT Regression Models

b) **1) Suggest a single best regression model (DT-1 or DT-2)** based on your 'USED' performance metric/s scores, which you defended in Task (3a). **2) Explain how your suggested model fulfils the success criteria.**

[4 Marks]

Selected Model: DT-1 (Fully Grown)

Justification:

- DT-1 achieved **lower MAE and MSE**, indicating smaller average and squared errors, and a higher R^2, showing better variance explanation.
- Since extreme values in Maximum Loan Amount are critical for risk management, the **fully grown tree captures these extremes better**, allowing more precise predictions for high-risk or high-value loans.
- DT-2, being pruned, reduces complexity but loses detail for extreme cases, which may underestimate or overestimate loans for clients at the tails of the distribution

Task (3) – Evaluating your Maximum Loan Amount DT Regression Models

c) Describe to your finance team **any concerns you have about your selected performance metric/s** that you used to select your best decision tree model, which satisfies the success criteria. [200 words maximum] with in-text citation.

[4 Marks]

While MSE, MAE, and R^2 provide a strong foundation for evaluating DT-1, there are limitations to note:

1. **MSE's sensitivity to outliers**: MSE heavily penalizes extreme errors, which can sometimes exaggerate the importance of a few very large deviations rather than reflecting typical prediction performance. If extreme values are rare, this could skew model assessment (Hyndman & Koehler, 2006).
2. **MAE ignores directionality**: MAE reports the magnitude of errors but not whether predictions systematically over- or underestimate the maximum loan amount. In practice, overestimating may increase default risk, while underestimating may reduce potential returns.
3. **R^2 limitation**: R^2 shows overall variance explained but does not directly quantify prediction errors for extreme loans. A high R^2 can coexist with poor performance on critical outliers.

Conclusion: Although DT-1 satisfies the success criteria by minimizing both average and squared errors, the team should be cautious and monitor **predictions for extreme loan cases**, potentially supplementing evaluation with **quantile-specific metrics** or **visual inspection of residuals** for high-value loans.

References with in-text citation

Agrawal, R. (2025) *Know The Best Evaluation Metrics for Your Regression Model!*, Analytics Vidhya, 23 June. Available at: https://www.analyticsvidhya.com/blog/2021/05/know-the-best-evaluation-metrics-for-your-regression-model/ (Accessed: 13 April 2026).

Task (3) – Evaluating your Maximum Loan Amount DT Regression Models

d) Client 60256 was Approved a Loan. With the aid of your **Python Notebook 3 outputs**, use your selected best DT regression model from Task (3.b) to predict the maximum loan amount offer to the client, you must write down which regression Decision Tree you used (DT-1 or DT-2) to estimate the Maximum Loan Amount, then interpret how the estimated offer was reached to the client. Client 60256 attributes' values are in the following table: **[6 Marks]**

Variable Name	Value
ID	**60256**
Sex	**Female**
Age	**56 Years old**
Education Qualifications	**Unknown**
Income	**57,000**
Home Ownership	**Rent**
Employment Length	**15**
Loan Intent	**Medical**
Loan Amount	**25700**
Loan Interest Rate	**23%**
Loan-to-Income Ratio (LTI)	**10%**
Payment Default on File	**No**
Credit History Length	**35**
Loan Approval Status	**Approved**
Maximum Loan Amount	**?**
Credit Application Acceptance	**0**

The best regression model selected in Task (3-b) is DT-2 (Pruned Decision Tree, max_depth=4).
Predicted Maximum Loan Amount for Client 60256: £3,846.70
Interpretation of how the estimated offer was achieved:
Client 60256's attributes were passed through both Decision Tree models (DT-1 and DT-2) for comparison.
The tree performed splits based on key features such as:

- Income (£57,000)
- Credit history length (35 years)
- Current loan amount (£25,700) and LTI ratio (45.09%)
- Employment length (15 years), Age (56), and Loan Intent (Medical)
- Home Ownership (Rent) and no prior default on file

DT-1 (Fully Grown) navigated through up to 30 levels of splits and returned to a predicted value of £0.00. This is a result of severe overfitting — with 27,990 leaf nodes, DT-1 essentially memorized the training data. The client's feature combination landed in a leaf containing only high-risk or zero-loan training samples, making the prediction unreliable for real-world use.

DT-2 (Pruned, max_depth=4) limited splits to 4 levels, forcing the tree to learn broader, more generalizable patterns. The client reached a leaf node whose training samples had an average max_allowed_loan of £3,846.70. This reflects a conservative lending decision for a client with Rent ownership, medical loan intent, and a LTI ratio of 45.09%, which the model associates with a moderate-to-high risk profile.
DT-2 is therefore the recommended model, as it generalizes reliably to new, unseen clients while DT-1 fails due to overfitting.